GameFeed

Réseau social centré sur le jeu vidéo

Titre Professionnel Développeur Web et Web Mobile — Niveau 5

Formation DWWM · 2025-2026

Plan de la présentation :

01 **Expression des besoins**
Contexte et objectifs du projet

02 **Environnement technique**
Stack, outils et versionning

03 **Réalisations Front-end**
Maquettes, interfaces, dynamisme

04 **Réalisations Back-end**
BDD, API REST, sécurité

05 **Jeu d'essai**
Scénario de test fonctionnel

06 **Veille sécurité**
Vulnérabilités identifiées et corrections

07 **Conclusion**
Bilan et perspectives

01 — Expression des besoins :

Contexte

GameFeed est un réseau social dédié aux joueurs, développé durant la formation, leur permettant de partager leurs expériences de jeu sous forme de posts, de liker les contenus d'autres joueurs et de consulter les profils.

Partager

Publier des posts liés à un jeu spécifique

S'identifier

Créer un compte et gérer son profil

Interagir

Liker et voir les publications d'autres joueurs

Naviguer

Feed global et pages de profil utilisateur

Contraintes : Sécurité OWASP · Accessibilité RGAA · RGPD · Éco-conception · Responsive (mobile + desktop)

02 — Environnement technique :

Front-end

- React 19 (Vite)
- React Router DOM
- Axios
- CSS vanilla
- ESLint (qualité code)

Back-end

- Node.js
- Express
- MySQL2
- bcrypt
- JSON Web Token

Outils & Méthodes

- Git + GitHub
- Thunder Client
- MySQL Workbench
- VS Code
- Méthode agile

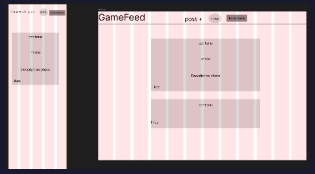
Versionning : Git avec commits séparés par fonctionnalité · GitHub public · .gitignore (node_modules, .env)

03 — Front-end · Maquettes :

Connexion

Formulaire sécurisé

Feed



Posts + navbar

Créer un post

Formulaire + sélecteur jeu

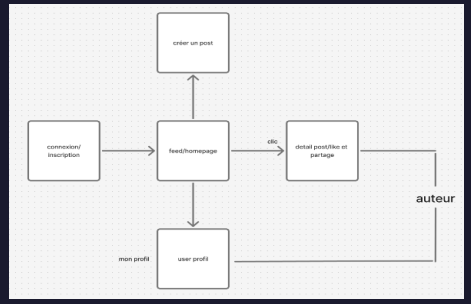
Profil

Stats + posts utilisateur

Modifier profil

Edition des infos

Schéma d'enchaînement :



03 — Front-end · Interfaces statiques :

HTML / CSS vanilla

Structure sémantique, classes réutilisables, pas de framework CSS

SEO basique

Balises meta, titres hiérarchiques, attributs alt

Responsive Design

Adaptation desktop et mobile via media queries (REAC : responsivité)

RGPD

Pas de données sensibles stockées côté client, token JWT en localStorage

Accessibilité

Labels sur les formulaires, contrastes respectés, RGAA pris en compte

Éco-conception

Pas de dépendances CSS lourdes, images en URL externe

03 — Front-end · Interfaces dynamiques :

Fonctionnalités React

useState / useEffect
Gestion de l'état local et des effets de bord

React Router DOM
Navigation entre pages sans rechargement

Context API (AuthContext)
État global de connexion partagé dans toute l'appli

Axios + intercepteurs
Injection automatique du token JWT dans chaque requête

Likes en temps réel
Mise à jour du compteur sans rechargement de page

Suppression temps réel
Retrait immédiat du post du feed après suppression

Sécurité front-end

- Validation des entrées côté client
- Protection XSS : pas de dangerouslySetInnerHTML
- Token JWT stocké, non exposé dans l'URL
- Routes protégées (PrivateRoute)
- Redirection si non authentifié
- Pas de mot de passe affiché en clair

04 — Back-end · Base de données :

users
id (PK)
username (UNIQUE)
email (UNIQUE)
password_hash
avatar_url
bio
created_at

games
id (PK)
name
cover_url
genre

posts
id (PK)
user_id (FK)
game_id (FK)
title
content
image_url
created_at

likes
id (PK)
user_id (FK)
post_id (FK)
created_at
UNIQUE(user_id, post_id)

Relations : users 1→N posts · games 1→N posts · users N→N posts (via likes) · CASCADE ON DELETE activé

04 — Back-end · API REST :

Auth	Posts	Likes	Users	Games
POST /api/auth/register	GET /api/posts	POST /api/likes/:postId 🔒	GET /api/users/:id	GET /api/games
POST /api/auth/login	GET /api/posts/user/:id	DELETE /api/likes/:postId 🔒	PUT /api/users/:id 🔒	
	POST /api/posts 🔒	GET /api/likes/:postId/check 🔒		
	DELETE /api/posts/:id 🔒			

🔒 = authentification JWT requise

Architecture en couches

index.js	routes/	controllers/	services/	middleware/auth.js	config/db.js

04 — Back-end · Sécurité :

Hashage des mots de passe :

bcrypt avec 10 tours de salage —> jamais le mot de passe en clair en base de données

Validation des entrées :

Vérification des champs obligatoires avant toute opération en base de données

Authentification JWT :

Token signé avec secret JWT_SECRET, valable 24h, transmis dans le header Authorization

Contrôle d'accès :

Un utilisateur ne peut modifier ou supprimer que ses propres ressources (vérification user_id)

Requêtes paramétrées :

Utilisation de '?' (prepared statements) dans toutes les requêtes SQL —> protection anti-injections SQL

Variables d'environnement :

Credentials en .env exclu du versionning Git — jamais de secrets dans le code source

05 — Jeu d'essai :

Fonctionnalité testée : création d'un post et like

1

Données en entrée

POST /api/posts
Body : { title: 'Test TP', game_id: 1, content: 'Fonctionnel !' }
Header : Authorization Bearer <token>

2

Données attendues

Status 201 Created
Body : { message: 'Post créé avec succès', id: 3 }

3

Données obtenues

Status 201 Created ✓
Body : { message: 'Post créé avec succès', id: 3 } ✓

4

Analyse des écarts

Aucun écart constaté.
Le post apparaît immédiatement dans le feed.
Le like fonctionne sans rechargement de page.

06 — Veille sécurité :

Sources de veille : OWASP · CERT-FR · ANSSI · MDN Security

CRITIQUE	**Injection SQL** Vulnérabilité : Possible si les requêtes SQL utilisent la concaténation de chaînes	Correction : Utilisation de requêtes paramétrées (?) avec mysql2 — aucune concaténation dans le code
ÉLEVÉ	**XSS (Cross-Site Scripting)** Vulnérabilité : Injection de scripts malveillants via les champs de formulaire	Correction : React échappe automatiquement les données affichées — pas de dangerouslySetInnerHTML utilisé
ÉLEVÉ	**Broken Authentication** Vulnérabilité : Token JWT non sécurisé ou expirant trop tard	Correction : JWT signé avec secret fort, expiration 24h, vérification à chaque requête protégée
MOYEN	**Exposition de données sensibles** Vulnérabilité : Mot de passe stocké en clair ou secret exposé dans le code	Correction : bcrypt (10 tours) pour les mots de passe — variables sensibles dans .env exclu du Git

07 — Conclusion :

Points positifs

- Application fullstack fonctionnelle de bout en bout
- Likes et suppression en temps réel sans rechargement
- Architecture en couches claire (routes/controllers/services/BDD)
- Sécurité appliquée à chaque niveau (JWT, bcrypt, SQL paramétré)
- Code commenté et versionné sur GitHub
- Interface responsive desktop et mobile

Difficultés rencontrées

- Configuration de dotenv (conflit avec dotenvx)
- Gestion du port 3000 déjà occupé
- Synchronisation état liké/non liké au chargement
- Organisation des repos Git (mono-repo vs multi-repo)
- Nommage des fichiers sensible à la casse sur Windows

Perspectives : Système de commentaires · Recherche de jeux via API RAWG · Déploiement en production (HTTPS, Docker) · Tests unitaires Jest

Merci pour votre attention

Questions ?

GitHub : github.com/phildentaire/devoir-bilan-formation-GameFeed

Stack : React · Node.js · Express · MySQL · JWT